

Department of
Artificial Intelligence & Machine Learning

B.Tech III Semester
Academic Year 2026-
2027

Data Base Management Systems (2501IT05)

Capstone Stone Project Documentation



ADITYA UNIVERSITY

Surampalem, Kakinada District
Andhra Pradesh-533437

ADITYA UNIVERSITY

(Formerly Aditya Engineering College(A))

Aditya Nagar, ADB Road, Surampalem, Andhra Pradesh, India

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



CERTIFICATE

This is to certify that the DBMS Capstone Project report entitled “Library Management System” is being submitted by M. SRI RAMANA (Roll No. 25B11AI704), K. CHANDRU (Roll No. 25B11AI465), G. SUHAS (Roll No. 25B11AI384) and A. DATTA SRI SAI (Roll No. 25B11AI049) in

partial fulfillment of the requirements for the award of the B.Tech. degree in Artificial Intelligence and Machine Learning. The work documented in this report was carried out as part of the **Capstone Project** for the course **Database Management Systems (2501IT05)** during the academic year **2026–2027**, and records the database design, implementation, and documentation undertaken by the team.

Faculty Coordinator

Mrs. Gayatri Lakshmi Narasamba Vanguri
Department of AI & ML

Head of the Department

Dr. K. Naga Bhargavi, M. Tech, Ph.D.
Associate Professor & HOD

External Examiner

DECLARATION

We hereby declare that the DBMS Capstone Project entitled “Library Management System” is a genuine work carried out by us. The database design, SQL implementation, queries and documentation presented in this report are our own work, completed under the guidance of Mrs. Gayatri Lakshmi Narasamba Vanguri, Department of Artificial Intelligence and Machine Learning, Aditya University, Surampalem. We further declare that this work has not been submitted, in full or in part, for the award of any degree of this or any other educational institution.

Project Associates

M. SRI RAMANA (25B11AI704)

K. CHANDRU (25B11AI465)

G. SUHAS (25B11AI384)

A. DATTA SRI SAI (25B11AI049)

Date: 06 October 2026 Place:
Surampalem

ACKNOWLEDGEMENT

It is with immense pleasure that we express our indebted gratitude to our faculty coordinator Mrs. Gayatri Lakshmi Narasamba Vanguri, Department of Artificial Intelligence and Machine Learning, who guided us throughout the capstone project. We are thankful for the constant encouragement and valuable suggestions received at every review stage.

We are grateful to Dr. K. Naga Bhargavi, Head of the Department, for inspiring us and for arranging the facilities and resources needed for this project.

We extend our sincere thanks to the Management, Dr. M. Sreenivasa Reddy, Deputy Pro-Chancellor, Dr. M. B. Srinivas, Vice-Chancellor, and Dr. A. Ramesh, Pro Vice-Chancellor (Engineering & Sciences), for providing an excellent academic environment.

Not to forget, we thank the Teaching and Non-Teaching Staff, our parents and our friends, who have directly or indirectly supported us in completing this project on time.

Project Associates

M. SRI RAMANA (25B11AI704)

K. CHANDRU (25B11AI465)

G. SUHAS (25B11AI384)

A. DATTA SRI SAI (25B11AI049)

TABLE OF CONTENTS

Update the table of contents in Microsoft Word after opening this report, if required.

S.NO	CONTENTS	PAGE NUMBER
1	Abstract	01
2	Introduction	01
3	Problem Statement	01
4	Objectives	02
5	Tools & Technologies	02
6	DATABASE DESIGN	02
7	Database implementation	06
8	Modules & functionalities	06
9	Github repository & team responsibilities	06
10	Enhancements & Future improvements	18
11	Conclusion	20
12	References	20

1. ABSTRACT

The Library Management System is a database-driven application that automates daily library activities such as book and member registration, searching, issuing, returning, reservations, and fine management. It replaces error-prone manual registers with a normalized relational database, using primary and foreign keys and constraints to reduce redundancy and preserve integrity. SQL queries support availability checks, overdue analysis, and member borrowing history.

2. INTRODUCTION

2.1 Background

A library manages a large collection of books and serves many members. Books may have authors, publishers, categories, and multiple physical copies. Members borrow and return copies at different times, so the library must monitor due dates, availability, reservations, and fines. A relational database stores this related information in connected tables and retrieves it accurately using SQL.

2.2 Purpose of the Database

The database provides a centralized repository for library operations. Librarians can maintain catalogue and member information, record loans and returns, calculate fines, and generate reports. Members can search books, view availability, place reservations, and review borrowing history.

3. PROBLEM STATEMENT

Manual library management requires repeated writing and checking of records. It is difficult to determine copy availability, identify overdue loans, track borrowing history, calculate fines, and

update information consistently. Duplicate data and missing records can lead to inaccurate reports and poor service. The proposed system solves these problems through a normalized relational database with validation, relationships, constraints, and SQL operations for cataloguing, circulation, reservations, and reporting.

4. OBJECTIVES

- Create a centralized database for books, copies, authors, publishers, categories, members, staff, and transactions.
- Enable fast searching by title, ISBN, author, category, and availability.
- Record issue and return transactions with due dates and overdue fines.
- Maintain integrity using primary keys, foreign keys, unique constraints, checks, and defaults.
- Generate reports for inventory, overdue loans, circulation, categories, and member history.

5. TOOLS & TECHNOLOGIES USED

Category	Tool / Technology Used
Database Server	PostgreSQL
Query / Admin Tool	pgAdmin 4 / PostgreSQL CLI
Front-end	HTML, CSS, JavaScript or Python Flask
Programming	Python and SQL
IDE	Visual Studio Code
Operating System	Microsoft Windows
Version Control	Git and GitHub

6. DATABASE DESIGN

6.1 Identified Entities

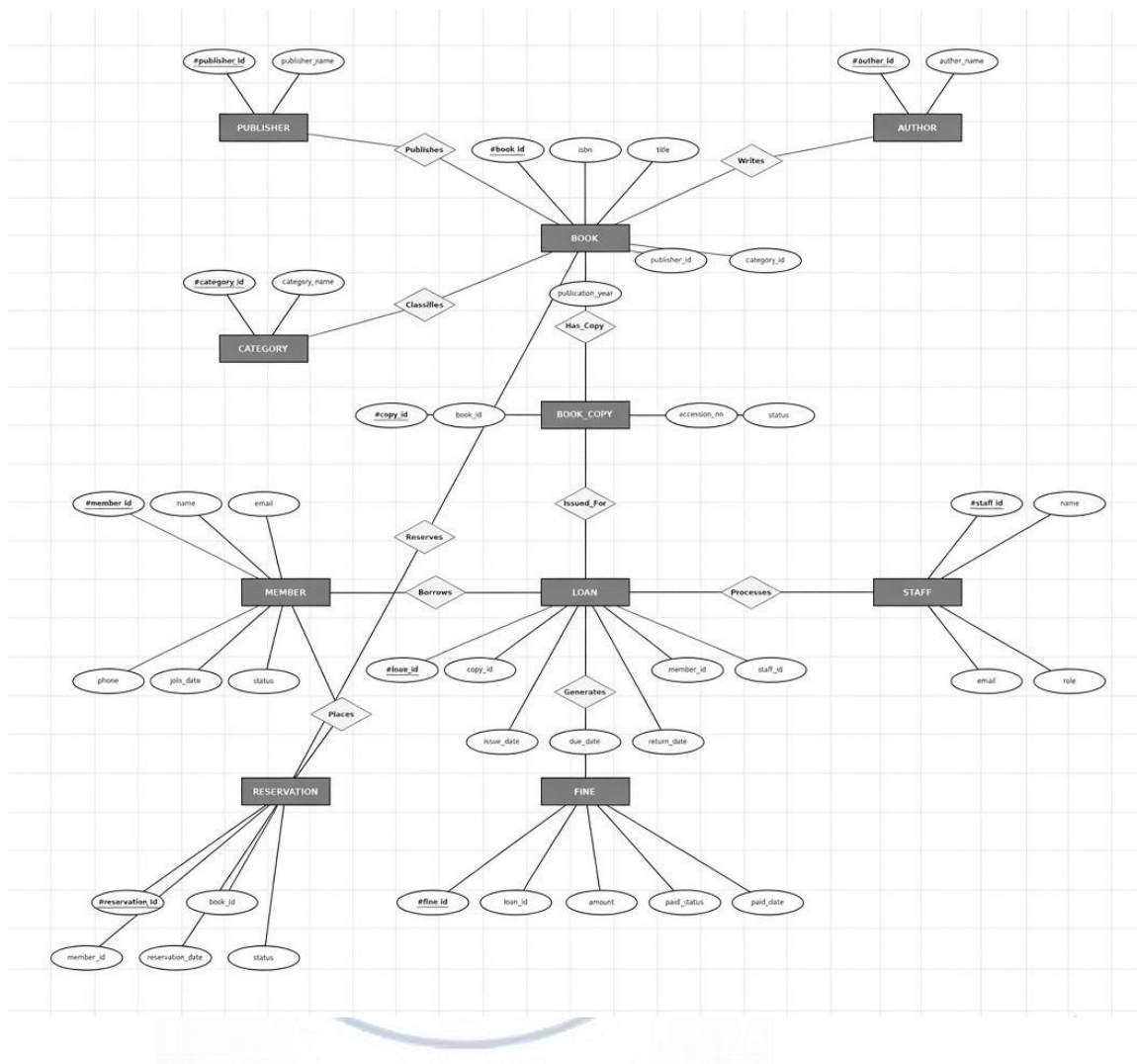
Entity	Description
Book	Logical bibliographic record with title and ISBN.
Book Copy	Physical copy that can be issued.
Author	Person who wrote a book.
Publisher	Organization publishing a book.
Category	Subject classification.
Member	Registered borrower.
Staff	Librarian or administrator.
Loan	Issue and return transaction.
Reservation	Request for a book.
Fine	Charge for an overdue loan.

6.2 Entity Attributes

Entity	Important Attributes	Keys / Notes
Book	book_id, isbn, title, publisher_id, category_id, publication_year	book_id PK; isbn UNIQUE; publisher_id/category_id FK
Book Copy	copy_id, book_id, accession_no, status	copy_id PK; book_id FK
Author	author_id, author_name	author_id PK
Publisher	publisher_id, publisher_name	publisher_id PK
Category	category_id, category_name	category_id PK
Member	member_id, name, email, phone, join_date, status	member_id PK; email UNIQUE
Staff	staff_id, name, email, role	staff_id PK
Loan	loan_id, copy_id, member_id, staff_id, issue_date, due_date, return_date	loan_id PK; FKs
Reservation	reservation_id, book_id, member_id, reservation_date, status	reservation_id PK; FKs
Fine	fine_id, loan_id, amount, paid_status, paid_date	fine_id PK; loan_id FK

ADITYA UNIVERSITY

6.3 ER Diagram



ADITYA UNIVERSITY

6.4 Relationships & Cardinality

Entity 1	Relationship	Entity 2	Cardinality
Publisher	publishes	Book	1:N
Category	classifies	Book	1:N
Book	has copies	Book Copy	1:N
Book	written by	Author	M:N
Member	borrows	Loan	1:N
Staff	records	Loan	1:N
Member	places	Reservation	1:N
Loan	generates	Fine	1:0..1

6.5 Relational Database Schema

Table	Primary Key	Foreign Keys	Constraints
publishers	publisher_id	—	name UNIQUE NOT NULL
categories	category_id	—	name UNIQUE NOT NULL
authors	author_id	—	name NOT NULL
books	book_id	publisher_id, category_id	isbn UNIQUE; title NOT NULL
book_authors	book_id + author_id	book_id, author_id	Composite PK
book_copies	copy_id	book_id	accession UNIQUE; status CHECK
members	member_id	—	email UNIQUE; status CHECK
staff	staff_id	—	role CHECK
loans	loan_id	copy_id, member_id, staff_id	date CHECKs
reservations	reservation_id	book_id, member_id	status CHECK
finances	fine_id	loan_id	amount CHECK; loan UNIQUE

6.6 Normalization

UNF: One register contains repeating book, author, member, and loan details. 1NF: Fields are atomic and repeating groups are separated. 2NF: The M:N book-author relation is moved to BOOK_AUTHORS. 3NF: Publisher, category, member, staff, loan, reservation, and fine details are separated so non-key attributes depend only on their table key. This reduces insertion, update, and deletion anomalies.

Constraint	Purpose
PRIMARY KEY	Unique row identification.
FOREIGN KEY	Referential integrity.
UNIQUE	Prevents duplicate ISBN, email, and accession numbers.
NOT NULL	Ensures required data.
CHECK	Restricts invalid statuses, dates, years, and amounts.
DEFAULT	Supplies consistent dates and initial statuses.

7. DATABASE IMPLEMENTATION

7.1 DDL - Database and Table Creation

IMPORTANT: Execute CREATE DATABASE separately first.

Step 1: Connect to the postgres database and run:

```
CREATE DATABASE library_management;
```

Step 2: Connect to the library_management database and execute the following code:

```
CREATE TABLE publishers (  
    publisher_id SERIAL PRIMARY KEY,  
    publisher_name VARCHAR(120) NOT NULL UNIQUE  
);
```

```
CREATE TABLE categories (  
    category_id SERIAL PRIMARY KEY,  
    category_name VARCHAR(80) NOT NULL UNIQUE  
);
```

```
CREATE TABLE authors (  
    author_id SERIAL PRIMARY KEY,  
    author_name VARCHAR(120) NOT NULL  
);
```

```
CREATE TABLE books (  
    book_id SERIAL PRIMARY KEY,  
    isbn VARCHAR(20) UNIQUE NOT NULL,  
    title VARCHAR(200) NOT NULL,  
    publisher_id INT NOT NULL REFERENCES publishers(publisher_id),  
    category_id INT NOT NULL REFERENCES categories(category_id),  
    publication_year INT CHECK (publication_year BETWEEN 1000 AND 2100)  
);
```

```
CREATE TABLE book_authors (  
    book_id INT NOT NULL REFERENCES books(book_id) ON DELETE CASCADE,  
    author_id INT NOT NULL REFERENCES authors(author_id) ON DELETE CASCADE,  
    PRIMARY KEY (book_id, author_id)  
);
```

```
CREATE TABLE book_copies (  
    copy_id SERIAL PRIMARY KEY,  
    book_id INT NOT NULL REFERENCES books(book_id),  
    accession_no VARCHAR(30) UNIQUE NOT NULL,  
    status VARCHAR(20) DEFAULT 'Available' CHECK (status IN ('Available', 'Issued', 'Lost',  
'Damaged'))  
);
```

```
CREATE TABLE members (  
    member_id SERIAL PRIMARY KEY,  
    name VARCHAR(120) NOT NULL,
```

```
email VARCHAR(150) UNIQUE NOT NULL,  
phone VARCHAR(20),  
join_date DATE DEFAULT CURRENT_DATE,  
status VARCHAR(15) DEFAULT 'Active' CHECK (status IN ('Active', 'Inactive'))  
);
```

```
CREATE TABLE staff (  
    staff_id SERIAL PRIMARY KEY,  
    name VARCHAR(120) NOT NULL,  
    email VARCHAR(150) UNIQUE NOT NULL,  
    role VARCHAR(20) CHECK (role IN ('Librarian', 'Admin'))  
);
```

```
CREATE TABLE loans (  
    loan_id SERIAL PRIMARY KEY,  
    copy_id INT NOT NULL REFERENCES book_copies(copy_id),  
    member_id INT NOT NULL REFERENCES members(member_id),  
    staff_id INT NOT NULL REFERENCES staff(staff_id),  
    issue_date DATE DEFAULT CURRENT_DATE,  
    due_date DATE NOT NULL,  
    return_date DATE,  
    CHECK (due_date >= issue_date),  
    CHECK (return_date IS NULL OR return_date >= issue_date)  
);
```

```
CREATE TABLE reservations (  

```

```

reservation_id SERIAL PRIMARY KEY,
book_id INT NOT NULL REFERENCES books(book_id),
member_id INT NOT NULL REFERENCES members(member_id),
reservation_date DATE DEFAULT CURRENT_DATE,
status VARCHAR(15) DEFAULT 'Pending' CHECK (status IN ('Pending', 'Fulfilled', 'Cancelled'))
);

```

```

CREATE TABLE fines (
    fine_id SERIAL PRIMARY KEY,
    loan_id INT UNIQUE NOT NULL REFERENCES loans(loan_id),
    amount NUMERIC(8,2) NOT NULL CHECK (amount >= 0),
    paid_status BOOLEAN DEFAULT FALSE,
    paid_date DATE,
    CHECK (
        (paid_status = FALSE AND paid_date IS NULL)
        OR (paid_status = TRUE AND paid_date IS NOT NULL)
    )
);

```

```

CREATE UNIQUE INDEX one_active_loan_per_copy
ON loans(copy_id)
WHERE return_date IS NULL;

```

```

CREATE VIEW current_loans AS
SELECT * FROM loans WHERE return_date IS NULL;

```

7.2 Sample Data - DML

```
INSERT INTO publishers(publisher_name) VALUES
```

```
('Pearson'),
```

```
('OReilly Media');
```

```
INSERT INTO categories(category_name) VALUES
```

```
('Computer Science'),
```

```
('Fiction'),
```

```
('Artificial Intelligence');
```

```
INSERT INTO authors(author_name) VALUES
```

```
('Ramez Elmasri'),
```

```
('Robert C. Martin'),
```

```
('Stuart Russell');
```

```
INSERT INTO books(isbn, title, publisher_id, category_id, publication_year) VALUES
```

```
('9780133970777', 'Database Systems', 1, 1, 2016),
```

```
('9780132350884', 'Clean Code', 1, 1, 2008),
```

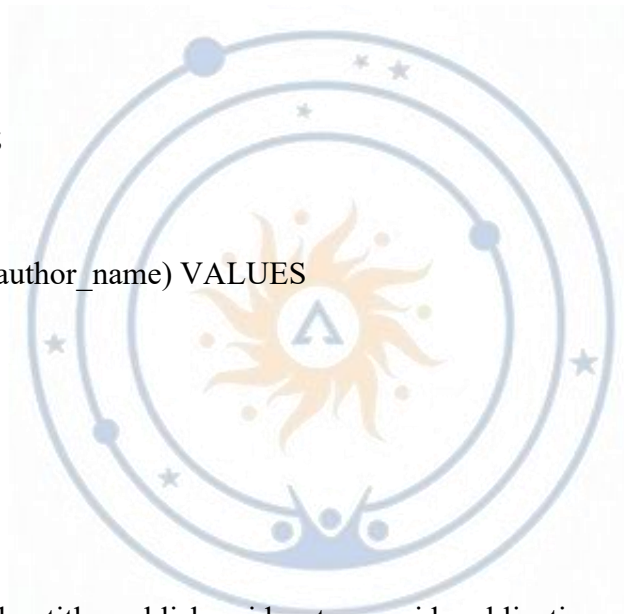
```
('9780134610993', 'Artificial Intelligence: A Modern Approach', 2, 3, 2021);
```

```
INSERT INTO book_authors VALUES
```

```
(1, 1),
```

```
(2, 2),
```

```
(3, 3);
```



ADITYA UNIVERSITY

```
INSERT INTO book_copies(book_id, accession_no) VALUES  
(1, 'ACC-001'),  
(1, 'ACC-002'),  
(2, 'ACC-003'),  
(3, 'ACC-004');
```

```
INSERT INTO members(name, email, phone) VALUES  
( 'Anil Kumar', 'anil@example.com', '9876543210'),  
( 'Priya Reddy', 'priya@example.com', '9876501234');
```

```
INSERT INTO staff(name, email, role) VALUES  
( 'Library Admin', 'admin@library.edu', 'Admin'),  
( 'S. Lakshmi', 'lakshmi@library.edu', 'Librarian');
```

```
INSERT INTO loans(copy_id, member_id, staff_id, due_date)  
VALUES (1, 1, 2, CURRENT_DATE + 14);
```

```
UPDATE book_copies
```

```
SET status = 'Issued'
```

```
WHERE copy_id = 1;
```

```
INSERT INTO reservations(book_id, member_id) VALUES
```

```
(3, 2);
```

7.3 Constraints & Data Integrity

Constraint	Purpose
PRIMARY KEY	Unique row identification.
FOREIGN KEY	Referential integrity.
UNIQUE	Prevents duplicate ISBN, email, and accession numbers.
NOT NULL	Ensures required data.
CHECK	Restricts invalid statuses, dates, years, and amounts.
DEFAULT	Supplies consistent dates and initial statuses.

7.4 SQL Operations & Queries

```
SELECT b.title, bc.accession_no
FROM books b
JOIN book_copies bc USING (book_id)
WHERE bc.status = 'Available';
```

```
SELECT m.name, b.title, l.issue_date, l.due_date
FROM loans l
```

```
JOIN members m USING (member_id)
```

```
JOIN book_copies bc USING (copy_id)
```

```
JOIN books b USING (book_id)
```

```
WHERE l.return_date IS NULL;
```

```
SELECT * FROM loans
```

```
WHERE return_date IS NULL
```

```
AND due_date < CURRENT_DATE;
```

```

SELECT c.category_name, COUNT(b.book_id) AS book_count
FROM categories c
LEFT JOIN books b USING (category_id)
GROUP BY c.category_name;

```

```

UPDATE loans
SET return_date = CURRENT_DATE
WHERE loan_id = 1;

```

```

UPDATE book_copies
SET status = 'Available'
WHERE copy_id = (
    SELECT copy_id
    FROM loans
    WHERE loan_id = 1
);

```



```

SELECT
    l.loan_id,
    m.name AS member_name,
    b.title,
    CURRENT_DATE - l.due_date AS overdue_days,
    (CURRENT_DATE - l.due_date) * 5.00 AS calculated_fine
FROM loans l
JOIN members m ON l.member_id = m.member_id
JOIN book_copies bc ON l.copy_id = bc.copy_id

```

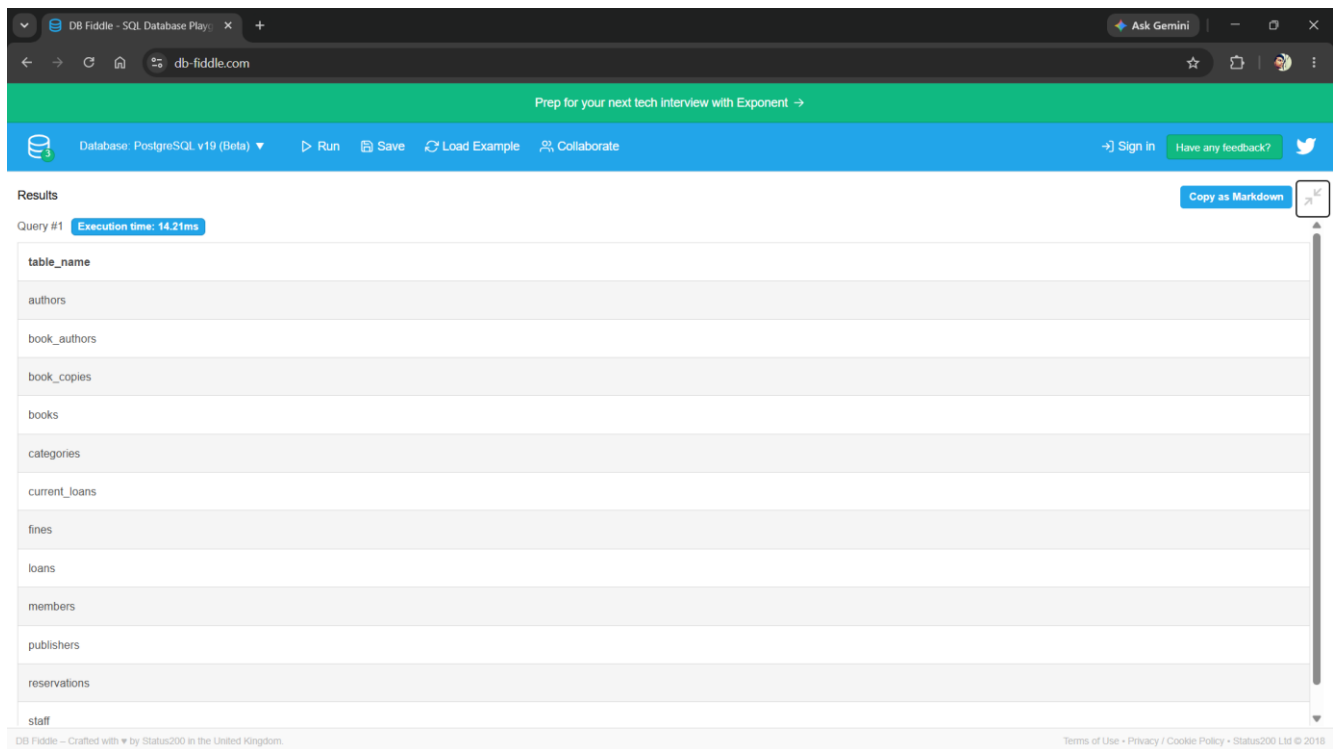
JOIN books b ON bc.book_id = b.book_id

WHERE l.return_date IS NULL

AND l.due_date < CURRENT_DATE;

7.5 Sample Outputs / Screenshots

Query 1-Shows all tables created



The screenshot shows the DB Fiddle web application interface. The browser tab is titled 'DB Fiddle - SQL Database Play'. The address bar shows 'db-fiddle.com'. The interface includes a green header with a navigation bar and a blue toolbar with buttons for 'Run', 'Save', 'Load Example', and 'Collaborate'. Below the toolbar, the 'Results' section displays the output of a query. The query is labeled 'Query #1' with an execution time of '14.21ms'. The results are presented as a table with one column, 'table_name', listing the following tables: authors, book_authors, book_copies, books, categories, current_loans, fines, loans, members, publishers, reservations, and staff. A 'Copy as Markdown' button is visible in the top right corner of the results area. The footer of the application states 'DB Fiddle - Crafted with ♥ by Status200 in the United Kingdom.' and provides links for 'Terms of Use', 'Privacy / Cookie Policy', and 'Status200 Ltd © 2018'.

table_name
authors
book_authors
book_copies
books
categories
current_loans
fines
loans
members
publishers
reservations
staff

ADITYA UNIVERSITY

Query 2-Show sample data from books table

DB Fiddle - SQL Database Playg...

Ask Gemini

db-fiddle.com

Prep for your next tech interview with Exponent →

Database: PostgreSQL v19 (Beta)

Run

Save

Load Example

Collaborate

Sign in

Have any feedback?

Results

Copy as Markdown

Query #1

Execution time: 0.25ms

book_id	isbn	title	publisher_id	category_id	publication_year
1	9780133970777	Database Systems	1	1	2016
2	9780132350884	Clean Code	1	1	2008
3	9780134610993	Artificial Intelligence: A Modern Approach	2	3	2021

DB Fiddle – Crafted with ♥ by Status200 in the United Kingdom.

Terms of Use • Privacy / Cookie Policy • Status200 Ltd © 2018

Query 3 - Show available books

DB Fiddle - SQL Database Playg...

Ask Gemini

db-fiddle.com

Prep for your next tech interview with Exponent →

Database: PostgreSQL v19 (Beta)

Run

Save

Load Example

Collaborate

Sign in

Have any feedback?

Results

Copy as Markdown

Query #1

Execution time: 0.58ms

title	accession_no	status
Database Systems	ACC-002	Available
Clean Code	ACC-003	Available
Artificial Intelligence: A Modern Approach	ACC-004	Available

DB Fiddle – Crafted with ♥ by Status200 in the United Kingdom.

Terms of Use • Privacy / Cookie Policy • Status200 Ltd © 2018

Query 4 -Show active loans

DB Fiddle - SQL Database Playg... x +

Ask Gemini

db-fiddle.com

Prep for your next tech interview with Exponent →

Database: PostgreSQL v19 (Beta) ▾

▶ Run

Save

Load Example

Collaborate

→ Sign in

Have any feedback?

Results

Copy as Markdown

Query #1 Execution time: 1.1ms

member_name	title	issue_date	due_date
Anil Kumar	Database Systems	2026-09-17	2026-10-01

Query 5 - Show category-wise book count

DB Fiddle - SQL Database Playg... x +

Ask Gemini

db-fiddle.com

Prep for your next tech interview with Exponent →

Database: PostgreSQL v19 (Beta) ▾

▶ Run

Save

Load Example

Collaborate

→ Sign in

Have any feedback?

Results

Copy as Markdown

Query #1 Execution time: 11.14ms

category_name	book_count
Computer Science	2
Artificial Intelligence	1
Fiction	0

8. MODULES & FUNCTIONALITIES

Module	Functionality
Authentication	Role-based librarian/admin access.
Book Catalog	Add, edit, delete, search, and display books.
Copy Management	Track physical copy status.
Member Management	Register members and update details.
Issue and Return	Record loans, dates, returns, and status changes.
Reservation	Manage requests for unavailable books.
Fine Management	Record overdue charges and payments.
Reports	Inventory, overdue, circulation, and category reports.

9. GITHUB REPOSITORY & TEAM RESPONSIBILITIES

Repository Link: [Sriramana-12/Library-Management-System: DBMS Capstone Project - Library Management System using PostgreSQL](https://github.com/Sriramana-12/Library-Management-System)

Student	Roll Number	Suggested Responsibility
M. Sri Ramana	25B11AI704	Coordination, integration, documentation
K. Chandru	25B11AI465	ER model, schema, normalization
G. Suhas	25B11AI384	DDL, DML, queries, testing
A. Datta Sri Sai	25B11AI049	UI, screenshots, GitHub organization

ADITYA UNIVERSITY

10. ENHANCEMENTS & FUTURE IMPROVEMENTS

The current Library Management System provides a robust database foundation. However, several enhancements can be implemented to transform it into a complete production-ready application:

10.1 User Interface Development

- Develop a responsive web application using HTML, CSS, JavaScript, and Python Flask or Django
- Create a mobile application using Flutter for Android and iOS platforms
- Design an intuitive dashboard for librarians with quick access to frequently used features
- Implement a member portal for self-service operations like searching books and viewing borrowing history

10.2 Security Enhancements

- Add secure user authentication with username and password login
- Implement password hashing using bcrypt or Argon2 for storing credentials securely
- Create role-based access control (RBAC) with different permissions for Admin, Librarian, and Member
- Add audit logs to track all critical operations like book issues, returns, and fine payments
- Implement session management and timeout for security

10.3 Notification System

- Send automated email or SMS reminders to members before due dates
- Notify members when their reserved books become available
- Alert librarians about low stock in popular categories
- Send overdue notices automatically after the due date passes

10.4 Advanced Features

- Integrate barcode or QR code scanning for quick book identification during issue and return
- Add ISBN lookup API integration to auto-populate book details
- Implement recommendation system suggesting books based on member's borrowing history
- Add support for e-books and digital resources management
- Enable bulk operations for importing book data from Excel or CSV files

10.5 Analytics and Reporting

- Create interactive dashboard charts showing circulation statistics, popular categories, and member activity
- Generate monthly and annual reports for library administration
- Add data visualization for borrowing trends, peak hours, and seasonal patterns
- Implement export functionality for reports in PDF and Excel formats

10.6 Performance and Scalability

- Add database indexes on frequently queried columns for faster search operations
- Implement connection pooling for better database performance
- Set up automated daily backups with point-in-time recovery capability
- Optimize queries and add caching for frequently accessed data
- Deploy on cloud platforms like AWS, Azure, or Google Cloud for scalability

10.7 Integration Capabilities

- Integrate with college ERP system for automatic member registration from student database
- Add payment gateway integration for online fine payments
- Connect with email services like SendGrid or SMTP for automated notifications
- Enable API endpoints for third-party integrations and mobile app connectivity

11. CONCLUSION

The Library Management System provides a structured solution for library data and operations. Its normalized schema, relationships, constraints, and SQL queries support cataloguing, members, circulation, reservations, fines, and reporting. The project demonstrates practical DBMS concepts and can be extended with a web or mobile interface.

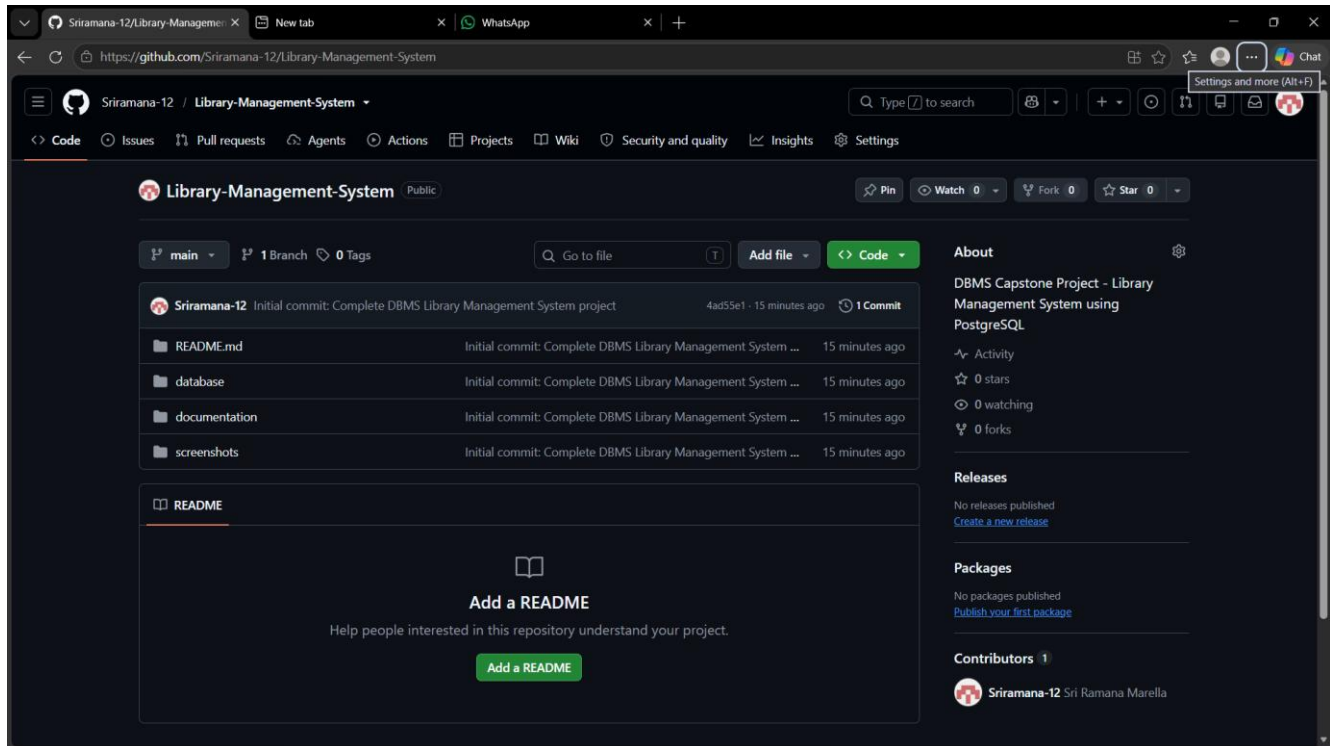
12. REFERENCES

1. Elmasri and Navathe, Fundamentals of Database Systems.
2. PostgreSQL Documentation.
3. Silberschatz, Korth, and Sudarshan, Database System Concepts.
4. Microsoft Word Documentation.

APPENDIX

A. Complete SQL Script

Save the DDL and DML from Section 7 as `library_management.sql` and upload it to GitHub. Add the final repository path here.



ADITYA UNIVERSITY